

Dec 10, 2025

CUDA:OpenCL translation

Invited Ayon Basumallik Peter Morales Niranjan Hasabnis Armand Mousavi
Matthew Freihofer Sanjna Ravichandar Tal Kadosh Bolet, Gregory Tomer Bitan
Le Chen galoren@technion.ac.il Avraham Evdaev Erel Kaplan
Soundararajan, Prema yosefgoren@campus.technion.ac.il le.chen@anl.gov
~~Octavian Udrea~~ ~~Laura Titolo~~ ~~Inbal Cohen~~

Attachments 📎 CUDA:OpenCL translation 📄 Notes - CUDA:OpenCL translation

Meeting records 🗒 Recording

Summary

Erel Kaplan presented the strong initial results for the Power Codex system using GPT codeex max, which successfully compiled, ran, and optimized code beyond human-written reference OpenMP code on HTC bench, Rodinia, and NASP parallel benchmarks in a local, non-internet access environment. Participants including Tom Yotam, Gal Oren, Le Chen, Niranjan Hasabnis, and Tomer Bitan discussed strategies to address potential data leakage by manipulating test cases, the need for an ablation study to justify the Power Codex framework's value beyond the base LLM, and the importance of showing separate results for correct translations and optimization achieved. The team, including Erel Kaplan, Le Chen, Tomer Bitan, Niranjan Hasabnis, and Tom Yotam, agreed on several actions including applying semantically preserving augmentations to benchmarks, documenting failed and successful translations, and structuring the paper to present benchmarks by increasing complexity to support the claims of the agentic framework.

Details

Notes Length: Standard

- **Introduction to Power Codex and Initial Results** Erel Kaplan presented the current results for ATC bench and lassi kernels, noting that they would also cover new use cases for power codex. The power codex system is an agentic pipeline utilizing OpenAI's codeex, which uses four instances guided by HPC expert instructions for code translation and optimization, including analysis, OpenMP pragmas, data movement, and offloading to the GPU. The system also features a correctness gate agent that compares the final state of main variables in the parallelized code against the serial version to ensure translation accuracy.
- **Power Codex Model Usage and Performance** Erel Kaplan initially used GPT codeex mini but encountered compilation failures on complex kernels, leading them to switch to the larger GPT codeex max model, which successfully compiled, ran, and even further optimized the code beyond the reference OpenMP code. Tom Yotam asked about the benchmarks and the correctness gate's reliance on golden examples from the data set, to which Erel Kaplan clarified that the correctness gate compares the offloaded version against the original serial code, not the OpenMP version from the benchmark suit.
- **Benchmarks and System Environment** Erel Kaplan confirmed that the system was tested on HTC bench, Rodinia, and NASP parallel benchmarks, which represent small, more complex, and real-world use cases, respectively. Gal Oren highlighted that Erel Kaplan is working in a non-internet access environment, meaning the codex system lacks external internet access and operates locally, which is critical because it prevents the model from searching for reference code or solutions. The benchmark choices were strategically made to allow comparison to existing literature, with NAS parallel benchmark and Rodinia being well-known HPC examples that produce meaningful results.
- **Addressing Data Leakage and Benchmark Integrity** Niranjana Hasabnis raised the concern that LLMs might have already seen the parallel code for common benchmarks in their training data and suggested adding a synthetic benchmark. Le Chen proposed manipulating test cases by changing variable names or deleting comments to make the code seem new to the model, which Gal Oren supported as a better option than a synthetic benchmark for tackling reviewer concerns about data leakage. Gal Oren further agreed that changing code

structure without changing functionality helps ensure the model is not over-fixated on specific points of the code.

- **Optimization and Performance Superiority** Tomer Bitan questioned the need for data leakage mitigation if the system's optimization results already beat the human-written reference baseline code. Gal Oren acknowledged this point but mentioned that reviewers might still argue that the model started the optimization from knowledge of the best solution. Tomer Bitan then suggested showing the intermediate step before optimization, where the translated code might perform worse, to demonstrate that the improvement comes from the optimization stage. Erel Kaplan agreed, noting that the first translation step was sometimes already good, but the second step utilized access to the profiler to resolve bottlenecks and perform micro-optimizations specific to the hardware.
- **Code Augmentation and Collaboration** Tom Yotam suggested using internal tools for semantically preserving augmentations on the code to prevent overfitting to specific kernels and syntax. Gal Oren requested Tom Yotam to apply this anonymization to the benchmarks and send them back to be included in the paper. Tomer Bitan and Niranjan Hasabnis further discussed that such modifications typically involve changing the Abstract Syntax Tree (AST), such as swapping operands in addition operations, while preserving semantics.
- **Prompt Engineering and Framework Justification** Le Chen inquired if the initial translation prompt contained specific instructions for generating efficient or high-performance code. Erel Kaplan clarified that the prompts provided the model with best practices for parallelizing the code. Gal Oren detailed that they provided the model with comprehensive guidance on OpenMP offloading and mapping of GPU and CPU resources to establish the "guided HPC expert" agent, which is akin to giving a student access to all parallel programming materials. Gal Oren further explained that power codex's value is not just better translation but showing that the model can run, compile, and optimize the code step-by-step using a multi-step thinking framework, which is necessary to mitigate all parallel code problems.
- **Reporting Failed Translations** Niranjan Hasabnis stressed the importance of splitting the results to show how many translations were correct and how much optimization was achieved. Gal Oren agreed and asked Erel Kaplan to provide two separate folders for each benchmark, containing the failed and successful translations, to analyze the logs and determine why some attempts failed. The

team redefined "failure" to include not just compilation issues, but also generating serial code or code with negative parallelization effects.

- **Benchmark Strategy and Prioritization** Gal Oren suggested organizing the paper's benchmark presentation by starting with Paraval, which defines common metrics for code translation, to address current literature. This would be followed by benchmarks like Paratron, Rodinia, and NAS, representing an escalation in complexity, with NAS being a massive, well-known HPC benchmark.
- **Access Bench Technical Issues** Erel Kaplan reported that the Accessbench benchmark was experiencing technical issues related to reconnecting to the model. Tomer Bitan speculated that this issue might be related to token caps or context limitations due to the kernel's large number of files and lengthy thought processes. Gal Oren considered this an engineering issue that could be mitigated quickly by changing deployment settings or extending token limits.
- **Interpreting Benchmark Results** Tomer Bitan explained the visual structure of the benchmark graph, where the top set of results uses the golden label's makefile and build environment, and the bottom set uses the model-created build environment. The results showed that the power codex system (labeled "codeex") was almost always better in terms of code score and overall score compared to other models.
- **Clarification on Codex Model and Framework** Tom Yotam questioned if the superior performance was due to the base LLM rather than the power codex framework, suggesting a need for an ablation study. Gal Oren clarified that the model used, GPT 5.1 Codex mini, is a model specifically trained for agentic command-line work. Gal Oren agreed that an ablation study comparing the model's baseline performance against the power codex framework's results is necessary to demonstrate the improvement offered by the agentic flow.
- **Outstanding Tasks and Documentation** Tomer Bitan confirmed they could share the modified benchmark repository with the team after creating a full branch. Gal Oren emphasized the need to share everything on Git, including readmes, and to ensure everyone has the necessary permissions for editing. Gal Oren also mentioned opening a new Discord channel for all project communication.
- **Final Results and Optimization Success** Erel Kaplan presented the Rodinia results using three metrics: kernel-only execution time, kernel plus memory transfers, and full execution time. The NAS results showed that power codex generally surpassed the reference, even though the zero-shot codex failed

compilation on the FT kernel. Erel Kaplan noted that for the FT and MG kernels, they had to switch to the larger GPT codeex max model. Gal Oren remarked that achieving such strong results, especially with the smaller GPT 5.1 mini model, is very encouraging and potentially a breakthrough in parallelizing serial code.

- **Conclusion and Path Forward** Gal Oren concluded the meeting by setting a one-month timeline for paper completion and emphasizing the need for extensive use of appendices to explain claims, add prompts, and include more detailed results. Niranjan Hasabnis complimented Erel Kaplan on the excellent work and effort .

Suggested next steps

- ☐ Erel Kaplan will share the GitHub repository of his results, including the prompts and everything, with the rest of the team and prepare two folders for each benchmark, separating the ones that failed and the ones that succeeded for further analysis.
- ☐ Tomer Bitan will prepare a full branch of the original repo with his modifications, push it to a shared repository, and add readmes so people can look into this as soon as possible.
- ☐ Tom Yotam will examine the benchmarks to see if the internal tools for semantically preserving augmentations for code apply, and will share his GitHub username in the Discord channel.
- ☐ Gal Oren will send Tom Yotam the benchmarks to be annotated with the internal tools for semantically preserving augmentations for code.
- ☐ The group will join the new Discord channel called 'Paracodex for everyone' and make sure everyone has access to the git and the overlift with editing permissions.

You should review Gemini's notes to make sure they're accurate. [Get tips and learn how Gemini takes notes](#)

Please provide feedback about using Gemini to take notes in a [short survey](#).